

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 1. oldal
--	--	---

Mikrokontroller alapú rendszerek

Elektronikus jegyzet

2. fejezet (2. rész)

Készítette: Dr. Tevesz Gábor c. egyetemi tanár
BME Automatizálási és Alkalmazott Informatikai Tanszék
1117. Budapest, Magyar tudósok körútja 2.
Q ép. B szárny II. em. B216.
Tel: 463-2881
Fax: 463-2871 (adm.)
Mail: tevesz@aut.bme.hu

Hallgatják: Villamosmérnöki és Informatikai Kar
Nappali tagozat
Villamosmérnöki alapszak (BSc)
III. évfolyam, 5. félév
Beágyazott és irányító rendszerek specializáció hallgatói

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 2. oldal
--	--	---

COPYRIGHT

Jelen dokumentum a BME Villamosmérnöki és Informatikai Kar hallgatói számára készített elektronikus jegyzet. A dokumentumot a Mikrokontroller alapú rendszerek c. tárgyat (BMEVIAUAC06) felvevő hallgatók jogosultak használni, és saját céljukra 1 példányban kinyomtatni. A dokumentum módosítása, bármilyen eljárással részben vagy egészben történő másolása tilos, illetve csak a szerző előzetes engedélyével történhet.

Copyright © 2008-2023 / Dr. Tevesz Gábor

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 3. oldal
--	--	---

TARTALOMJEGYZÉK

2. HARDVERKÖZELI PROGRAMOK FEJLESZTÉSE	4
2.18 Számábrázolási formátumok	4
2.18.1 Az egész típus	6
2.18.2 A valós típus	10
2.19 Esettanulmány: egy szoftver rendszerterv elemei	20

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 4. oldal
---	---	--

2. HARDVERKÖZELI PROGRAMOK FEJLESZTÉSE

2.18 Számábrázolási formátumok

Ebben a fejezetben áttekintjük azokat a legfontosabb számábrázolási formátumokat, melyeket a mikrokontrollerekben hardver vagy szoftver műveletvégző egységeink használnak. A hardverközeli programozás során gyakran térünk át a különböző számábrázolási formátumok között (sokszor egy egyszerű kasztolással), lényeges tehát pontosan ismernünk, hogy a kezelt adategységekben mely bitszoportok milyen célt szolgálnak ill. befolyásolnak – hasonlóan fontos ismernünk a számábrázolási formátumok pontosságát, és korlátait.

A mikrokontrollerekben a műveleteket alapvetően a számok kétféle nagy csoportján kívánjuk elvégezni: sok esetben elegendőek a számításokhoz az egyszerűbb, egész típusú (**integer**) számok, de sokszor szükségünk van a nem egész (tört) fogalom kezelésére is – ezen kívül a számok nagysága vagy a pontosság megkívánja a valós (**real**) számok alkalmazását. Szintén a törtszámok alkalmazása szükséges, ha matematikai függvényeket alkalmazunk (négyzetgyök, szinusz, stb.) programrendszerünk algoritmusában. Az egész számokkal való műveletvégzés általában gyorsabban elvégezhető, az egész aritmetikát valamennyi mikrokontrollernek támogatnia kell, ezért sokszor használjuk őket még törtszámok kifejezésére is úgy, hogy a számba “beleképzelünk” valahova egy kettedes (bináris) pontot, és ennek mozgását a műveletvégzések során állandóan egyszerű eltolási műveletekkel korrigáljuk (ún. **fixpontos számábrázolás**). Ezzel a módszerrel – bizonyos korlátok között – át lehet hidalni a felbontás és a pontosság kérdését, de nem segít azon az alapvető gondon, hogy a számok nagysága igen tág határok között változhat, és az ábrázolás pontosságát leginkább a mindenkorai tényleges számnagysághoz kellene igazítanunk (tehát az abszolút „1/100” pontosság semmitmondó a szám nagyságának ismerete nélkül). Ezt a matematikában a számok normál alakjával hidaljuk át, ennek a megfelelője a számítástechnikában az ún. **lebegőpontos számábrázolás**.

Az első mikroprocesszorok – melyek alapvetően 8 bites mikroprocesszorok voltak – többnyire még hardver szorzó- és osztó egységgel sem rendelkeztek, ezeknél bizony még problémát jelentettek akár a nagyobb egész számokkal való alapl műveletek is. Műveleteink operandusai akkor sem voltak “kisebbek”, mind ma: a 8 vagy 16 bites ábrázolásból adódó -128..+127 ill. -32768..+32767 számtartomány legtöbbször nem elegendő számítási műveletsoraink elvégzéséhez. Aritmetikai szubrutincsomagok végezték a nagyobb (32 bites) számok összeadását, kivonását, nem is beszélve a szorzási vagy a még időigényesebb osztási műveletekről (ld. 1.7 fejezet).

A 16/32 bites mikroprocesszorok megjelenése nagyot változtatott ezen az állapoton, különösen, ha figyelembe vesszük azt, hogy a szóhossz növekedése mellett megjelentek a processzorok utasításkészletében az (egész, de már előjeles) szorzó-osztó műveletek (ld. 68000 vagy 80x86). Ezekkel az egész típusú műveletvégzési problémáink zöme meg is oldódott.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 5. oldal
---	---	--

A lebegőpontos számokkal végzendő műveletek számítása még alpműveletek esetén is igen komoly időigénnyel jár, nem beszélve a függvények többnyire polinomiális sorba fejtett alakjainak közelítő számításairól. Ezekre a feladatokra speciális hardver áramköröket fejlesztettek ki, melyek a processzorok “társaként” az ilyen típusú műveletvégzésekre specializálódtak. Ezek a **társprocesszorok** (ún. coprocesszorok) a lebegőpontos aritmetikai műveletvégzés mellett egész típusú műveleteket is el tudtak végezni – 32, sőt 64 biten – áthidalandó a már említett hossz- és szorzási/osztási problémákat. Ez a tulajdonságuk azonban a korszerűbb 16/32 bites mikroprocesszorok megjelenésével egyre inkább háttérbe szorult.

A törtszámok kezelése egy “külön gondolatvilágot” alkot a számítógépek világában. Alkalmazásuk során ugyanis olyan problémák lép(het)nek fel, melyek az egész típusú műveletvégzés során eszünkbe sem jutnak. Gondoljuk meg ugyanis, hogy

- a törtszámokat gondolatban a matematika valós számfogalmával azonosítjuk. Mivel azonban bármely számítógépben egy törtszámot csak **véges számú biten** tudunk ábrázolni, az ábrázolásnak van egy véges pontossága és számábrázolási nagysága – és itt mindkettő véges voltán van a hangsúly, ami a matematikai valós szám fogalmában nincs benne.
- jól megszokott **véges törtrészű** tizedes törtek a 2-es számrendszerben való ábrázolásból adódóan nem biztos, hogy véges bitszámon ábrázolhatók kettedes törtként is (nem ugyanazok lesznek a „kerek” törtszámaink, mint pl. az 1/10).
- a számítógépek azon legnagyobb erénye, hogy a műveletvégzésük igen gyors az ember műveletvégzési sebességéhez képest, arra ösztönözte felhasználóit, hogy olyan számításokat végeztessenek vele, amely műveletek millióit vagy milliárdjait jelenti – „kivárható” véges idő alatt. Igen ám, de amikor minden egyes műveletvégzésünk elvileg sem pontos (a számábrázolás pontatlansága miatt), milliányi műveletet tartalmazó és egymásra épülő műveletsorok végén a végeredmények pontatlansága már akkora lehet, ami korábban az emberek számára elképzelhetetlen volt – mivel több emberélet is kevés lett volna az adott műveletsor elvégzésére. Megjelent a **numerikus labilitás** fogalma (egy bizonyíthatóan konvergens matematikai műveletsor kimenete azért kezd oszcillálni, mert számábrázolásunk és műveletvégzésünk pontossága nem elegendő).

Az elmondottak miatt a lebegőpontos számok kezelését (mantissza és karakterisztika mérete, ábrázolási formája, pontossági és kerekítési előírások, stb.) egyre inkább “uniformizálták”, míg 1985-ben az amerikai IEEE (Institut of Electrical and Electronics Engineers) **IEEE Std. 754-1985** néven szabványosította azokat (másik ismeretes elnevezése IEC 60559:1989, Binary floating-point arithmetic for microprocessor systems). Általában aki ma lebegőpontos számokról beszél – legyen az PC vagy mikrokontroller –, ehhez a szabványhoz tartja magát (eltekintve olyan kivételektől, mint amiket a jelfeldolgozó processzoroknál tanultunk). A szabvány kiterjesztése **IEEE Std. 754-2008** néven jelent meg, ez megtartotta az eredeti formátumokat, de hozzáigazította az ábrázolási méreteket a mai buszméretekhez (128 bit a korábbi 80/96 bit helyett).

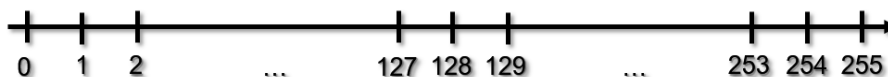
BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 6. oldal
--	--	---

2.18.1 Az egész típus

Az egész számok mindig vagy előjel nélküli, vagy előjeles (kettes komplementes kódú) számábrázolással értendők, az ábrázolható számok a következők:

Bits	Name	Range	Decimal digits
8	byte, octet	Signed: -128 to +127 Unsigned: 0 to +255	3 3
16	halfword, word	Signed: -32,768 to +32,767 Unsigned: 0 to +65,535	5 5
32	word, doubleword, longword	Signed: -2,147,483,648 to +2,147,483,647 Unsigned: 0 to +4,294,967,295	10 10
64	doubleword, longword, long long, quad, quadword	Signed: -9,223,372,036,854,775,808 to +9,223,372,036,854,775,807 Unsigned: 0 to +18,446,744,073,709,551,615	19 20
128		Signed: -170,141,183,460,469,231,731,687,303,715,884,105,728 to +170,141,183,460,469,231,731,687,303,715,884,105,727 Unsigned: 0 to +340,282,366,920,938,463,463,374,607,431,768,211,455	39 39

Már az egész számoknál is fontos kiemelni, hogy ábrázolásuk minden esetben (a rendszert tervező mérnök döntésétől függő) adott bitszámon történik, ami egy nem elfelejthető jelenséget hoz maga után. Vegyük az egyszerűség kedvéért az egy bájt (8 biten) ábrázolható pozitív egész számokat ! Mindenki tudja fejből, hogy 8 biten 256 különböző számértéket tudunk ábrázolni, 0-val kezdve ezek a 0..255 számok. Egy szokásos számegyenesen ábrázolva ezt:



Ábrázolható pozitív egész számok 1 bájt (8 biten)

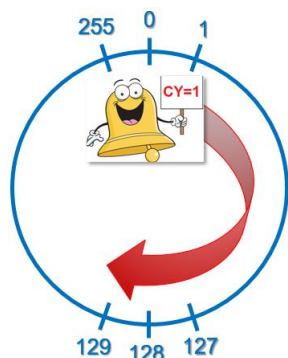
A számegyenes azonban mindkét irányban végtelen, megszokott matematikai ismereteink „becsapnak” a tartomány határain, hiszen a 255-ös értékhez hozzáadva 1-et mindenki azonnal rávágna, hogy az eredmény 256. Pedig nem így van, a 256 nem ábrázolható 8 biten, az összeadás után ez eredmény „túlcsordul” és az előbb említett

$$256_{10} = 1\ 0000\ 0000_2$$

eredményből csak az alsó 8 bit marad meg, azaz 0 lesz az eredmény és a mikrokontroller státuszregiszterében a CY=1 flag hívja fel a figyelmünket arra, hogy az eredmény túlcsordult, az eredményregiszterben kapott számérték „nem helyes”.

Az adott bitszámon ábrázolható számértékek fentiek miatt sokkal szemléletesebben ábrázolhatók a számegyenes helyett egy kör kerületén („feltekertük a számegyeneset”), mivel a körvonal véges volta miatt a 255-ös végérték a 0-s kezdőérték szomszédja lesz, így senki sem fog csodálkozni a következő (természetesen matematikailag helytelen) kifejezéseken:

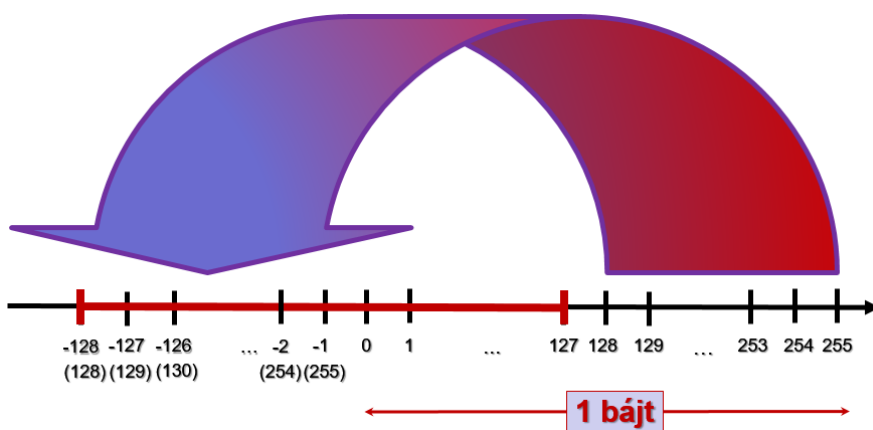
BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 7. oldal
--	--	---



$$\begin{aligned}
255 + 1 &= 0 & (\text{és } CY &= 1) \\
0 - 1 &= 255 & (\text{és } CY &= 1)
\end{aligned}$$

A számábrázolásunk véges voltát a lehetséges értékek egy körvonalon való elhelyezése jobban mutatja

Még bonyolultabb a helyzet az előjeles egész számok körében. Ismert mindenki számára, hogy ilyenkor a rendelkezésre álló tartomány felében ábrázoljuk a pozitív számokat (8 biten a 0..127 tartományban), a tartomány másik fele a negatív számoké (-1..-128). Az aritmetikát támogató kettes komplement számábrázolás az eredeti tartomány felső felét „transzformálja” a számegyenesen a negatív számok tartományába az alábbi ábra szerint:



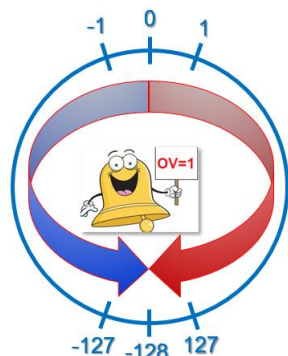
Ábrázolható előjeles egész számok 1 bájt (8 biten)

Itt is igaz viszont az adott bitszámon ábrázolható számok véges volta, csak a túlsordulás máshova kerül: a $0 \leftrightarrow 255$ (kettes komplement értelmezésben a $0 \leftrightarrow -1$) átmenet teljesen természetes és üzemszerű, semmilyen túlsordulás esete sem forog fent. Hol van ilyenkor az „átfordulás”, a túlsordulás? Jól látható, hogy a $127 \leftrightarrow 128$ -as értékek között, hiszen kettes komplement kódban a 128 már a -128-as értéket reprezentálja!

Akkor most itt kellene jeleznie a CY státuszbitnek? Jól legyenek meg: mikrokontrollerünknek fogalma sincs arról, hogy mi a számokat előjeles vagy előjel nélküli értelmezéssel fogjuk kiolvasni! A 128-as számérték pontosan ugyanazt a bitkombinációt jelenti, mint kettes komplement értelmezésben a -128, tehát mikrokontrollerünk nem tud máshogyan reagálni, mint hogy a $0 \leftrightarrow 255$ átmenetnél állítja továbbra is a CY bitet 1-be. Előjeles számoknál viszont ez egy „téves” jelzés, hiszen a

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 8. oldal
--	--	---

túlcsordulás valójában a $127 \leftrightarrow -128$ átmenetnél következik be. Ezt egy másik státuszbit, az OV (Overflow) jelzi számunkra a mikrokontroller státuszában:

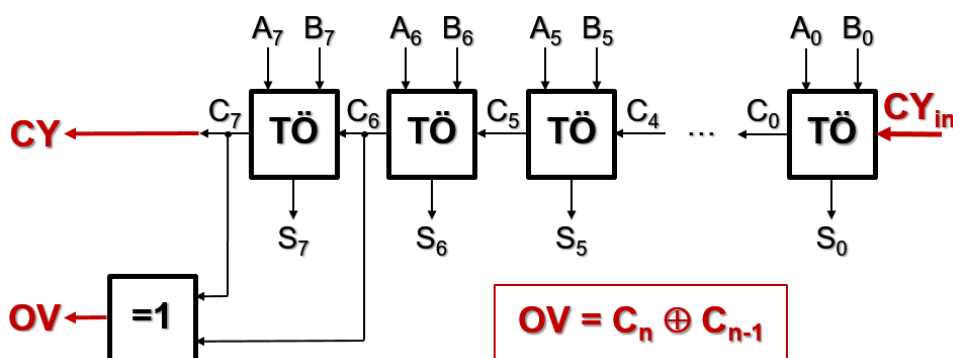


$$127 + 1 = -128 \quad (\text{és } OV = 1)$$

$$-128 - 1 = 127 \quad (\text{és } OV = 1)$$

A számbázisunk véges voltát a lehetséges értékek egy körvonalon való elhelyezése ebben az esetben is jobban mutatja

A két státuszbit előállítását rendkívül egyszerű (a Digitális technikában tanult egy bites teljes összeadó kapcsolást ismertnek feltételezve:



Ábrázolható előjeles egész számok 1 bájt (8 biten)

Miért rendkívül fontos erről tudnunk? Mert a jelzések figyelembevétele a programozó döntése kell legyen (milyen feltétel szerint ágazik el a program) aszerint, hogy hogyan értelmezi az adott regiszterben található biteket: előjeles vagy előjel nélküli számokként. A mikrokontroller mit sem tud erről, ha rossz feltételt vizsgálunk, a lehető legnehezebben felderíthető hibát építjük be a programunkba: az esetek többségében az algoritmus jól fog működni (mikor nincs túlcsordulás), néha pedig nem (amikor van).

A megjegyzendő alapszabály tehát:

**Előjel nélküli számokkal végzett aritmetikai műveleteknél a CY bit,
előjeles számokkal végzett aritmetikai műveleteknél az OV bit
jelzi a túlcsordulást.**

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 9. oldal
--	--	---

Az egész számok kezelésénél még meg kell említsük a számábrázolási méretek közötti áttérést (pl. 8 bit helyett 16 biten ábrázoljuk a számokat – ilyen pl. a típuskonverzióknál fordul elő, vagy perifériák kezelésénél, ha a 10 vagy 12 bites előjeles A/D értéket 16 biten akarjuk ábrázolni, mivel 8 biten nem fér el.)

A számábrázolási hossz növelése esetén ún. **előjel-kiterjesztést** kell alkalmaznunk. Ne tévesszük ugyanis szem elől, hogy kettes komplement számábrázolásunk bitszámhoz kötött fogalom ($2^n - A$), azaz az átalakítás során igazodnunk kell ahhoz, hogy a számot hány biten ábrázoljuk.

Pozitív számok esetén az áttérés egyszerű: megszoktuk, hogy egy szám elé tetszőleges számú 0-t írva a szám értéke nem változik.

Negatív számok esetében gondoljuk végig a következőt: $N > n$ bitesre szeretnénk átírni A számunkat. Ekkor igazak a következő összefüggések:

$$x = 2^n - |A| \quad \text{ill.} \quad y = 2^N - |A|$$

azaz y értékét szeretném meghatározni x-ből.

$\Leftarrow N \text{ bit} \Rightarrow$		
11111111	11111111	= $2^N - 1$
	11111111	= $2^n - 1$
11111111	00000000	= $(2^N - 1) - (2^n - 1)$
$\Leftarrow n \text{ bit} \Rightarrow$		

Mivel

$$y = 2^N - |A| = 2^N - 2^n + 2^n - |A| = (2^N - 1) - (2^n - 1) + 2^n - |A|$$

$$y = (2^N - 1) - (2^n - 1) + x$$

Azaz: negatív szám esetén a változatlan szám elejét egyesekkel kell feltöltenünk. Mivel egy szám pozitív, legfelső (előjel-) bitjének értéke mindig 0, negatív szám esetén ez 1, a fenti szabályt általánosíthatjuk:

Előjel-kiterjesztés esetén a szám elejéről hiányzó bitek a szám saját előjelbitjének értékével töltendők fel.

A számábrázolási hossz csökkentése esetén fentiek alapján egyszerűen megfogalmazható a fenti szabály fordítottja:

A szóhossz csökkentése esetén a szám elejéről egyszerűen elhagyhatjuk (levághatjuk) a legfelső biteket mindaddig, amíg
a) a levágott bitek azonosak (csak 0-k vagy 1-ek) ÉS
b) megegyeznek a lerövidült szám elején szereplő bittel

Fentiekre néhány példa:

	8 biten	12 biten	16 biten
+5 =	0000 0101	0000 0000 0101	0000 0000 0000 0101
-5 =	1111 1011	1111 1111 1011	1111 1111 1111 1011
+20 =	0001 0100	0000 0001 0100	0000 0000 0001 0100
-20 =	1110 1100	1111 1110 1100	1111 1111 1110 1100

2.18.2 A valós típus

A valós számokkal való műveletvégzéshez fix- vagy lebegőpontos számábrázolási formátumot használunk.

Fixpontos számábrázolás során az ábrázolt számba „beleképzelünk” egy ketteses pontot, és onnan kezdve a számmal ugyanúgy végezzük a műveleteket, mint az egész számokkal. A helyiértékeken balról jobbra a súlyozó faktorok folyamatos feleződés nem csak az egész részre igaz ($16 - 8 - 4 - 2 - 1$), hanem folytatódik a törtészben is ($1/2 - 1/4 - 1/8$ stb.) Egy 16 biten ábrázolt, 8 bites egészrész és 8 bites törtész tartalmazó fixpontos törtszámot úgy képzeljük el, mintha a szám helyett az ő 256-szorosát ábrázoltuk volna, de értelmezéskor a számot gondolatban elosztjuk 256-tal a ketteses pont odaképzésével.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}
I ₇	I ₆	I ₅	I ₄	I ₃	I ₂	I ₁	I ₀	F ₇	F ₆	F ₅	F ₄	F ₃	F ₂	F ₁	F ₀

•
ketteses pont

Az így ábrázolt számokon végzett műveletek minden további nélkül helyesek lesznek: ha két 16 bites számnak (A és B) tekintjük a fenti számokat, akkor az

$$A/256 + B/256 = (A+B)/256$$

értelmezés szerinti művelet is helyes eredményt ad. Ugyanez igaz előjeles számok esetére is:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}
S	I ₆	I ₅	I ₄	I ₃	I ₂	I ₁	I ₀	F ₇	F ₆	F ₅	F ₄	F ₃	F ₂	F ₁	F ₀

•
ketteses pont

Ebben az esetben az előbbi művelet:

$$(2^N \cdot A)/256 + (2^N \cdot B)/256 = (2^{N+1} \cdot A - B)/256$$

(Ne zavarjon meg senkit a 2^N tag 2^{N+1} -re változása, az csak a kettes komplement számábrázolás üzemzerű túlszorzásának eredménye.)

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 11. oldal
--	--	--

Példa 1:

8 biten ábrázolt 4 egész + 4 törtbitet tartalmazó szám esetén

$$00100100 = 0010.0100_2 = 2.25_{10}$$

A megoldandó feladat:

$$5 - 2.25 = 2.75$$

A szokásos 2-es komplementes képzéssel

$$-2.25 = 11011100$$

Végezzük el a műveletet, mintha egész számokkal dolgoznánk:

$$\begin{array}{r}
01010000 \quad (+5.00) \\
+ 11011100 \quad (-2.25) \\
\hline
00101100
\end{array}$$

Ez pedig a konvencióink szerint

$$0010.1100_2 = 2 + \frac{1}{2} + \frac{1}{4} = 2.75$$

Példa 2:

Oldjuk meg a következő műveletet

$$100 - 3.14 = 96.86$$

16 biten ábrázolt 8 egész + 8 törtbitet tartalmazó fixpontos számokkal.

Először is alakítsuk át a decimális számokat bináris megfelelőjükké !

Minden lépésben 2-vel osztva

Minden lépésben 2-vel szorozva

100	0	LSB
50	0	
25	1	
12	0	
6	0	
3	1	
1	1	MSB
0		

0.14	0.28	0	MSB
0.28	0.56	0	
0.56	1.12	1	
0.12	0.24	0	
0.24	0.48	0	
0.48	0.96	0	
0.96	1.92	1	
0.92	1.84	1	LSB

$$100_{10} = 1100100.00000000_2 \quad 3.14_{10} = 00000011.00100011_2$$

Jól látható a számaábrázolás véges pontossága, mivel

$$00000011.00100011_2 = 3.13671875_{10}$$

A szokásos 2-es komplementes képzéssel

$$-3.14_{10} = 11111100.11011101$$

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 12. oldal
--	--	--

(Szokás szerint jobbról balra haladva az első 1-ig mindent változatlanul hagyunk, onnan kezdve minden az ellenkezőjére változik. Ebben a műveletben a ketteses pontot figyelmen kívül hagyjuk, az a fixpontos számot ábrázoló regiszterben sincs ott.)

Végezzük el a műveletet, mintha egész számokkal dolgoznánk:

$$\begin{array}{r}
01100100.00000000 \quad (+100.00) \\
+ 11111100.11011101 \quad (-3.14) \\
\hline
01100000.11011101 \quad (+96.86)
\end{array}$$

Az egészrész egyszerűen ellenőrizhető:

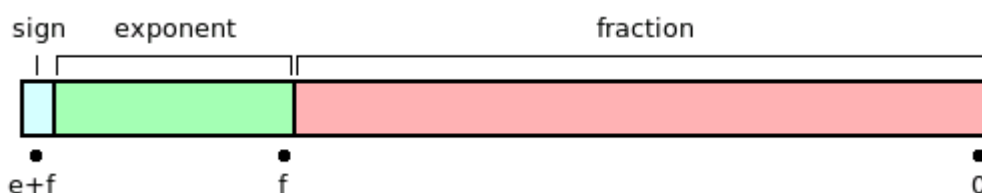
$$64 + 32 = 96,$$

a törtrész pedig - ne bajlódjunk a sok kettő hatvány reciprokával, léptessük gondolatban 8 helyiértékkel balra a számot (egész), majd léptessük ugyanennyit jobbra (/256) :

$$(255-32-2) / 256 = 221 / 256 = 0.86328125$$

A fixpontos számábrázolás az egyszerűsége miatt rendkívül gyors (szinte az egész számokéhoz hasonló) műveletvégzést tesz lehetővé. Arra kell nagyon vigyázni, hogy a magasabb rendű műveletek (szorzás, osztás) esetén a ketteses pont visszakerüljön a helyére (ugyanis az első példa szerinti 4+4 bites ábrázolásban a 16-szoros számérték szorzás után már 256-szoros lenne, azaz a számot „vissza kell tolnunk” 4 bittel jobbra, hogy a számábrázolás helyes legyen). Elsősorban a jelprocesszorok körében elterjedt a fixpontos ábrázolás és az ezekhez kapcsolódó aritmetikai könyvtárak.

A lebegőpontos számok a matematikai normál alak fogalmához kapcsolódnak. A világ többnyire igazodik a bevezetőben hivatkozott IEEE-754 szabvány (sajnos néhol már elavult) előírásaihoz. A számok ábrázolása itt a következőképpen néz ki:



A lebegőpontos számábrázolás általános alakja

A szám értelmezése pedig:

$$x = \pm m \cdot 2^k$$

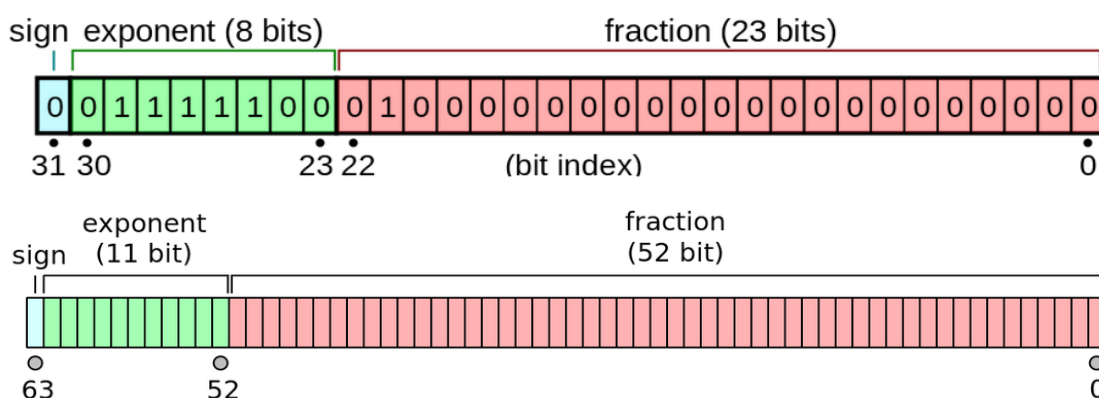
(Már a tárgyalás elején felhívjuk az olvasó figyelmét arra, hogy a fenti képletben szereplő **m** mantissza nem azonos az ábrázolásban „fraction” névvel jelölt törtrésszel, ugyanígy a **k** karakterisztika értékét tekintve nem azonos az „exponent” mező értékével.)

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 13. oldal
--	--	--

A **mantissza** az $[1.0 \dots 2.0)$ balról zárt intervallumba normált szám együttható része. Ezt az alakot a valós szám ún. normalizált valós alakjának nevezzük. Az elkülönülő előjel miatt a mantissza mindig pozitív. Mivel a mantissza egész része az intervallumból adódóan mindig 1, így ezt a szabvány előírásának megfelelően nem is ábrázoljuk (ún. rejtett 1-es). A *fraction* mezőben ábrázolt mantissza (f db bit) tehát valójában a tényleges mantissza törtrésze ($m=1.\text{törtrész}$).

Az *exponens* (e db bit) mindig egy pozitív szám, a "valódi" kitevő (karakterisztika) 0 értékéhez a mező méretéből adódó félérték–1 (pl. 8 bit esetében 127) tartozik. Ezt a nullponteltolást a továbbiakban offsetnek nevezzük (offset = n jelöléssel a kapott kódolást n többletes kódnak nevezzük). Az exponens ábrázolásából adódó minimális (0) és maximális értékeket (pl. \$FF) speciális célokra tartották fent (ld. később).

A szabvány többféle lebegőpontos típust (méretet) definiál:



Az IEEE-754 szabvány szerint értelmezett lebegőpontos számábrázolások

Fentiekén kívül az 1985-ös szabványban még egy kiterjesztett (extended) pontosság is értelmezett, ez viszonylag ritkábban használatos igen terjedelmes mérete (12 bájt = 96 bit) miatt. A tényleges számábrázolás itt 80 biten történik, mivel azonban a 10 (bájt) nem osztható 4-gyel, a 32 bites szóhosszúsághoz igazodva a szám $3 \times 4 = 12$ bájton való ábrázolását írja elő a szabvány. A 2008-as revízió ezen ábrázolási forma helyett már a 128 bites ábrázolási formátumot vezeti be.

	Előjel	Exponens	Törtrész	Összesen
Single	1	8	23	32
Double	1	11	52	64
Extended	1	15	64	80 (+16 bit nem használt)
128 bit	1	15	112	128

Látható, hogy a szabvány 2008-as revíziója nem a számábrázolás nagyságát növelte meg (az exponens mező szélessége változatlan maradt), hanem a pontosságát (a törtrész mezőhossza nőtt). A tényleges számábrázolási határokat később még látni fogjuk, azt azonban már itt is leszögezhetjük, hogy a 15 bites exponens ($2^{16382} \approx 10^{4932}$) által lehetővé tett számábrázolási nagyságot nem sok értelme volna tovább növelni – viszont azon el

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 14. oldal
--	--	--

kell gondolkoznunk, hogy a 23 bites törtrész ~ 7 , az 52 bites ~ 16 , a 64 bites ~ 20 , a 112 bites pedig ~ 34 hasznos számjegy ábrázolását teszi lehetővé. Ezen pontosságok értékelésénél mindig azt tartjuk szem előtt, hogy alkalmazott számítási algoritmusunk milyen pontatlanság halmozását eredményezheti – szélsőséges esetben egy egymillió műveletet tartalmazó ciklusos iteráció után a 6-7 számjegy kiindulási pontosságú operandusunk már csak $10^{-7} * 10^6 = 10\%$ pontosságú !

Külön kell szólni a számábrázolás speciális eseteiről: a denormalizált alakról, a 0, a $\pm \infty$ ábrázolásáról és az ún. nem definiált alakról.

SM	Exponens (e)	Törtrész (f)	Értelmezés	Elnevezés
0/1	$0 < e < \text{Max}$	Tetszőleges	$\pm 1.f \cdot 2^{e - \text{off}}$	Normalizált szám
0/1	0	nem 0	$\pm 0.f \cdot 2^{0 - \text{off} + 1}$	Denormalizált szám
0/1	0	0	$\pm 0.0 \cdot 2^{0 - \dots}$	Egzakt 0
0/1	Max	0	-	$\pm \infty$
0/1	Max	nem 0	-	NaN (Not-a-Number)

Számtartományok értelmezése az IEEE-754 szabvány szerint

A denormalizált alak szerepe igen lényeges: minden számábrázoláshoz egy véges nagyságú ún. felbontás rendelhető. Ez az a legkisebb “lépcső”, amivel az adott pontosságú szám növelhető vagy csökkenthető, értéke értelemszerűen függ a számábrázolás pontosságától (S/D/X/128), pl. egyszeres pontosságú számábrázolás esetén

$$\text{Felbontás} = 1.00..0 * 2^{1-127} = 1.2 * 10^{-38}$$

Ezzel a pontossággal “együtt kell élnünk”, hiszen az általunk választott számábrázolás következménye. Igazán kritikussá a dolog a 0 közelében válik, hiszen ott a 0 és az ábrázolható legkisebb szám (=felbontás) között nincs további érték, a számábrázolás hibája végtelen nagygyá válhat. Vegyük azonban észre, hogy “lenne még lehetőség” a számban pontosításra – ha feladjuk azt az alapelvet, hogy a számnak a [1.0 .. 2.0) balról zárt intervallumba kell normálva lennie: a

$$\text{Felbontás} = 0.10..0 * 2^{1-127} (=0.10...0 * 2^{0-126})$$

szám még mindig ábrázolható, sőt az 1-es elvihető a mantissa legvégéig is; ezzel növeltük a pontosságot, de megszűnt az eddig alapszabályként őrzött ún. rejtett 1-es.

A denormalizált alak tehát a 0 közelében a felbontás pontosságát növeli azzal, hogy áttér az [1.0...2.0) intervallumról a (0.0...1.0) intervallumra (így tehát még a törtrész

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 15. oldal
--	--	--

helyiértékeinek számával növelhető a pontosság – csökkentve ezzel viszont az ábrázolt szám hasznos jegyeinek számát). Nézzük a fizikailag ábrázolható számokat pl. az egyszeres pontosságú (single) számábrázolási formátumnál:

Sign	Exp	Frac	Mant	Kar	Típus	s	e	f
0	11111111	11111111111111111111111111111111	1.11..11	$\cdot 2^{(255-127)}$	NaN	0/1	Max	$\neq 0$
0	11111111	11111111111111111111111111111110	1.11..10	$\cdot 2^{(255-127)}$				
		...						
0	11111111	00000000000000000000000000000010	1.00..10	$\cdot 2^{(255-127)}$				
0	11111111	00000000000000000000000000000001	1.00..01	$\cdot 2^{(255-127)}$	+ ∞	0/1	Max	=0
0	11111111	00000000000000000000000000000000	1.00..00	$\cdot 2^{(255-127)}$				
		...						
0	11111110	11111111111111111111111111111111	1.11..11	$\cdot 2^{(254-127)}$	Normalizált számok	0/1	$0 < e < \text{Max}$	bármí
0	11111110	11111111111111111111111111111110	1.11..10	$\cdot 2^{(254-127)}$				
		...						
0	00000010	00000000000000000000000000000001	1.00..01	$\cdot 2^{(2-127)}$				
0	00000010	00000000000000000000000000000000	1.00..00	$\cdot 2^{(2-127)}$				
0	00000001	11111111111111111111111111111111	1.11..11	$\cdot 2^{(1-127)}$				
0	00000001	11111111111111111111111111111110	1.11..10	$\cdot 2^{(1-127)}$				
		...						
0	00000001	00000000000000000000000000000001	1.00..01	$\cdot 2^{(1-127)}$				
0	00000001	00000000000000000000000000000000	1.00..00	$\cdot 2^{(1-127)}$				
0	00000000	11111111111111111111111111111111	0.11..11	$\cdot 2^{(0-126)}$	Denormalizált számok	0/1	= 0	$\neq 0$
0	00000000	11111111111111111111111111111110	0.11..10	$\cdot 2^{(0-126)}$				
		...						
0	00000000	00000000000000000000000000000010	0.00..10	$\cdot 2^{(0-126)}$				
0	00000000	00000000000000000000000000000001	0.00..01	$\cdot 2^{(0-126)}$	Egzakt 0	0/1	= 0	= 0
0	00000000	00000000000000000000000000000000	0.00..00	$\cdot 2^{(0-126)}$				

Számtartományok egyszeres pontosságú számábrázolás esetén

Vegyük észre azt a nagyon fontos tény, hogy a normalizáltról a denormalizált tartományra történő áttérésnél az exponens offsetje 1-gyel csökken (a mantissza egésze nincs ábrázolva!).

Fentiek alapján valamennyi típusra értelmezhetjük az ábrázolható legnagyobb szám, a normalizált és a denormalizált felbontás, ill. hasznos számjegyek fogalmát (utóbbit természetesen csak a normalizált alak esetében):

Típus	Ábrázolható legnagyobb szám	Normalizált felbontás	Denormalizált felbontás	Hasznos (dec.) jegy
Single	$3.4028 \cdot 10^{+38}$	$1.1754 \cdot 10^{-38}$	$1.4012 \cdot 10^{-45}$	6-7
Double	$1.7976 \cdot 10^{+308}$	$2.2250 \cdot 10^{-308}$	$4.9406 \cdot 10^{-324}$	15-16
Extended	$1.1897 \cdot 10^{+4932}$	$3.3621 \cdot 10^{-4932}$	$3.6451 \cdot 10^{-4951}$	19-20
128 bit	$1.1897 \cdot 10^{+4932}$	$3.3621 \cdot 10^{-4932}$	$1.2950 \cdot 10^{-4965}$	33-34

A NaN-típus vagy hibás számmegadás útján jöhet létre, vagy a lebegőpontos műveletvégző egységek generálják meghatározott esetekben. Itt kell megjegyeznünk, hogy a végtelen "definit" használata nem okoz problémát a műveletek számára, a szabvány előírásai szerint, pl.:

Művelet	Eredmény
$n / \pm\infty$	0
$\pm\infty * \pm\infty$	$\pm\infty$
$\pm[\text{nemnulla}] / 0$	$\pm\infty$
$\infty + \infty$	∞
$\pm 0 / \pm 0$	NaN
$\infty - \infty$	NaN
$\pm\infty / \pm\infty$	NaN
$\pm\infty * 0$	NaN

Nézzünk néhány példát a fentiek könnyebb megértésére (a könnyebb áttekinthetőség érdekében csak az egyszeres pontosságú számábrázolást használjuk):

Példa 1:

Legyen $x = 5$. Ez a szám az $[1..2)$ intervallumba normálva: $x = +1.25 * 2^2$.

- a keletkező szám előjele $+$ ($=0$),
- az exponens értéke 2 ($+127=129$),
- a mantissa binárisan 1.01_2 , amelyből a rejtett 1-es levágása után $.01_2$ marad.

s	exponens								tötrész																							
0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
1. bájt								2. bájt								3. bájt								4. bájt								

Példa 1: $x = 5$ egyszeres pontosságú ábrázolása (0x40A00000)

Példa 2:

Legyen $x = -0.75$. Ez a szám az $[1..2)$ intervallumba normálva: $x = -1.5 * 2^{-1}$.

- a keletkező szám előjele $-$ ($=1$),
- az exponens értéke -1 ($+127=126$),
- a mantissa binárisan 1.1_2 , amelyből a rejtett 1-es levágása után $.1_2$ marad.

s	exponens								tötrész																				
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1. bájt								2. bájt								3. bájt								4. bájt					

Példa 2: $x = -0.75$ egyszeres pontosságú ábrázolása (0xBF400000)

Példa 3:

Legyen $x = 1/3$. Ez a szám az $[1..2)$ intervallumba normálva: $x = +4/3 \cdot 2^{-2}$.

(Segítség: $1/3 - 1/4 = 1/12$, $1/12 - 1/16 = 1/48$, $1/48 - 1/64 = 1/192$, $1/192 - 1/256 = 1/768 \dots$)

- a keletkező szám előjele $+$ ($=0$),
- az exponens értéke -2 ($+127=125$),
- a mantissza binárisan $1.010101\dots_2$, amelyből a rejtett 1-es levágása után $.010101\dots_2$ marad.
- a véges bitszámból visszaszámított pontos eredmény: $x = 0.3333333134651184$

s	exponens								tötrész																						
0	0	1	1	1	1	1	1	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0						
1. bájt								2. bájt								3. bájt								4. bájt							

Példa 3: $x = 1/3$ egyszeres pontosságú ábrázolása (0x3EAAAAA)

Példa 4:

Legyen $x = 2014$. Először alakítsuk át bináris számmá pl. a tanult felezési módszerrel (a számot osztjuk 2-vel, a hányados lesz a következő osztandó, a maradék pedig a bináris szám legkisebb helyiértékű bitje – a folyamatot addig ismétljük, míg 0 eredményt nem kapunk):

Osztandó/hányados	Maradék	Helyiérték
2014	0	LSB
1007	1	
503	1	
251	1	
125	1	
62	0	
31	1	
15	1	
7	1	
3	1	
1	1	MSB
0		

$$\text{Tehát } 2014 = 11111011110 = 1.1111011110 \cdot 2^{10}$$

(Segítség: 2 kitevője megegyezik az eredetileg a szám végén található ketteses pont balra léptetéseinek számával.)

- a keletkező szám előjele $+$ ($=0$),
- az exponens értéke $+127+10 = 137$ (az egyszerűbb átalakítás érdekében: $128+9$)
- a mantissza binárisan 1.111101111 (a legelső 0-t nem írtuk ki)

s	exponens								tötrész																						
0	1	0	0	0	1	0	0	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0		
1. bájt								2. bájt								3. bájt								4. bájt							

Példa 4: 2014 egyszeres pontosságú ábrázolása (0x44FBC000)

Példa 5:

Legyen $x = 2.014$ (A példa tanulsága: mit is okoz egy egyszerű decimális eltolás ?)

Először is alakítsuk az egész és a törtrészt is bináris számmá. A szám egész része ránézésre megmondható (nagyobb számok esetén a 6. példában alkalmazott felezéses módszert alkalmazzuk), a törtrészt pl. a tanult kétszerezési módszerrel számoljuk át. (Most az átváltandó törtrészt szorozzuk 2-vel, az eredményként kapott szám egészrésze lesz a keresett bináris törtrész legmagasabb helyiértékű bitje, a kapott törtrész pedig a következő lépés szorzandója. A művelet sor akkor hagyható abba, ha a törtrész 0-nak adódik – innen kezdve minden további bit 0 –, vagy ha a lépések száma elérte az ábrázolni kívánt bináris törtrész szükséges bitjeinek számát.)

Törtrész	Egészrész	Helyiérték
0.014	0	törtrész MSB
0.028	0	
0.056	0	
0.112	0	
0.224	0	
0.448	0	
0.896	1	
0.792	1	
0.584	1	
0.168	0	
0.336	0	
0.672	1	
0.344	0	
0.688	1	
0.376	0	
0.752	1	
0.504	1	
0.008	0	
0.016	0	
0.032	0	
0.064	0	
0.128	0	törtrész LSB

Tehát $2.014 = 10.000000111001010110 = 1.0000000111001010110 \cdot 2^1$

- a keletkező szám előjele + (=0),
- az exponens értéke $+127+1=128$
- a mantissza binárisan $1.0000000111001010110\dots$ (ez még folytatódna !)

s	exponens								törtrész																							
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	1	0	1	0	1	1	0	0	0	0			
1. bájt									2. bájt								3. bájt								4. bájt							

Példa 5: 2.014 egyszeres pontosságú ábrázolása (0x4000e560)

A 4. és az 5. példa eredményének összevetése jól mutatja, hogy egy szám decimális eltolása (10 hatványával való szorzása vagy osztása) teljesen átstrukturálja a bináris számot, nem csak az egyes bitek változnak meg, de a szám is átalakul „kerek” törtrészből akár végtelen szakaszos kettedes törtté.

Példa 6:

Legyen x = a minimális ábrázolható pozitív szám (denormalizált alak)

- a keletkező szám előjele + (=0),
- az exponens értéke -126 ($k=0$, a denormalizált alak miatt az offset 1-gyel csökken),
- a mantissza binárisan 0.00...1 (nincs rejtett 1-es, 0 nem lehet, mert az a 0 szám)
- $x = 2^{-126} \cdot 2^{-23} = 2^{-149} = 1,4013 \cdot 10^{-45}$

s	exponens								tötrész																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Példa 6: A legkisebb szám egyszeres pontosságú ábrázolása (0x00000001)

Példa 7:

Legyen x = a maximális ábrázolható pozitív szám

- a keletkező szám előjele + (=0),
- az exponens értéke +127 (+127=254), 255 nem lehet, mert a maximum a ∞ lenne
- a mantissza binárisan 1.11...1 (=2 minimális kerekítési hibával)
- $x = 2 \cdot 2^{+127} = 2^{+128} = 3,4028 \cdot 10^{+38}$

s	exponens								tötrész																							
0	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1		
1. bájt									2. bájt								3. bájt								4. bájt							

Példa 7: A legnagyobb szám egyszeres pontosságú ábrázolása (0x7F7FFFFFFF)

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 20. oldal
--	--	---

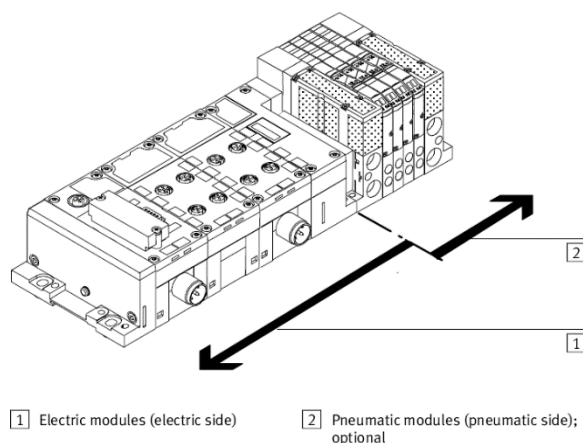
2.19 Esettanulmány: egy szoftver rendszerterv elemei

Ebben a fejezetben egy ténylegesen lezajlott gyártmányfejlesztési folyamat első lépéseit mutatjuk be, azokat a lépéseket, mikor a fejlesztő team a feladat megértésétől, megfogalmazásától kezdődően lebontja annak elemeit hardver és szoftver által megoldandó részekre, felállítja a szoftver rendszer vázát és elkészíti az ún. szoftver rendszertervet.

Bármely feladatban a firmware előállítása „egyszeri” befektetés, szemben a minden példányban újra és újra megépítendő hardver részegységekkel a szoftver úton megvalósított megoldás már csak minimális újabb költségekkel (szoftver karbantartása, javítása, update) járó reprodukciót jelent. Érthető tehát, hogy különösen azokban a jelfeldolgozási feladatokat megkövetelő ipari irányítórendszerekben, melyekben a változtatható (paraméterezhető) tulajdonságokkal rendelkező jelfeldolgozást bonyolult állapotgépeket feltételező hardver kapcsolások helyett szoftver által megvalósított programozott funkciók tehermentesítik a hardver úton megvalósítandó feladatokat, fokozottan oda kell figyelniük a feladatok szétosztására, a hardver és a szoftver szoros kapcsolatára.

Bármilyen hihetetlennek tűnik is, a szoftverrel történő jelfeldolgozás egyik legkomolyabb ellenfele a mai napig az idő, ill. a sebesség: egyszerűnek tűnő részfeladatok nagy tömegben és nagy sebességgel szoftver úton történő elvégzésére óvatosan, előzetes számolások alapján vállalkozzunk csak. Ne értsük félre a problémát: nem a mikrokontrollerek maximális teljesítőképességével, sebességével van gondunk, hanem egy költségoptimált megoldást kell előállítanunk: „túl nagy” kontroller kiválasztása drágább megoldást jelenthet, mint egy egyszerű elemekkel realizálható hardver megoldás – az optimum megtalálása a fejlesztő igazi felelőssége.

Az ipari automatizálási elemeket gyártó Festo cég egy gyártmánycsaládjának (CPX) alapvető felépítése a következő: a terepi buszra (Profibus DP) illeszkedő lokális érzékelő/beavatkozó egységek modulárisan, tetszőleges konfigurációban egymás mellé építhetők, egy-egy ún. ”sziget” kialakítása a következő:



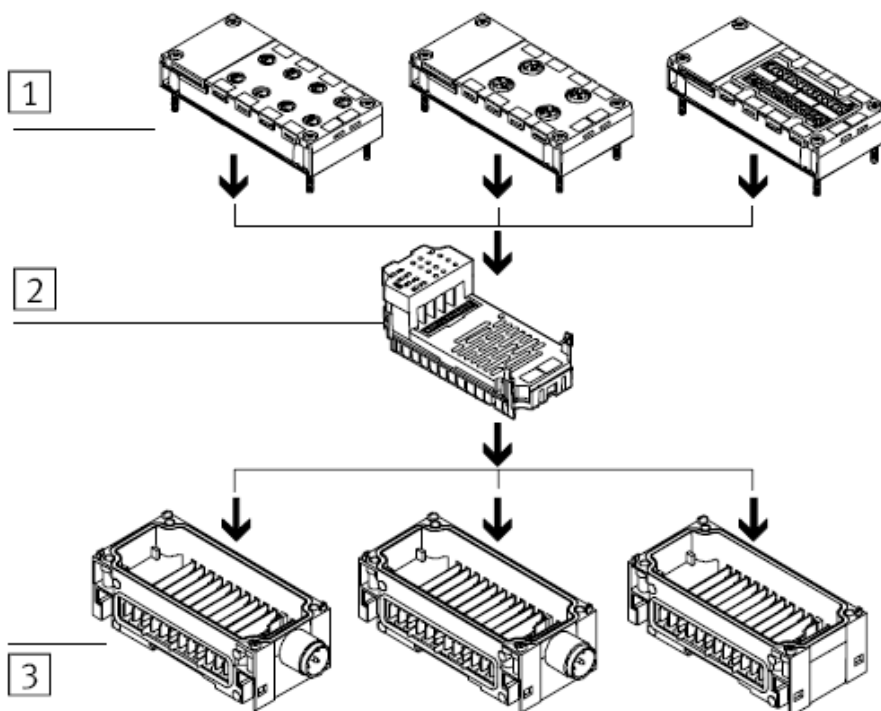
A Festo cég CPX folyamattírányító modulrendszere

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 21. oldal
--	--	--

A sziget baloldali részén az elektronikus modulok (a terepi busszal kommunikáló ún. master egység, digitális és analóg be- és kimeneti modulok) található, míg a jobb oldalon a cég fő profiljához illeszkedő miniatűr pneumatikus végrehajtó egységek (kétállapotú és ún. arányszelepek) helyezkednek el, melyek vezérléséről szintén az elektronikus rész gondoskodik.

A rendszer teljesen modulárisan, a mindenkori igényeknek megfelelően építhető fel. A kiválasztott modulok egymás mellé helyezhetők, az oldalaikon található csatlakozórendszer biztosítja mind a modulok tápellátását (24V elektronika, 24V nagyobb terhelésekhez, 0V, PE), mind a modulok közötti kommunikációt. Utóbbi egy nagysebességű (1 Mbaud) soros buszon történik, melyet a cég saját protokollal látott el és speciális saját kommunikációs áramkör (CBUS ASIC) gondoskodik az adatok ciklikus átviteléről a master és a modulok között. Egy láncoló (daisy-chain) logika gondoskodik a tetszőleges módon összeállított modulkonfiguráció bekapcsolás utáni felismeréséről és a modulok azonosíthatóságáról.

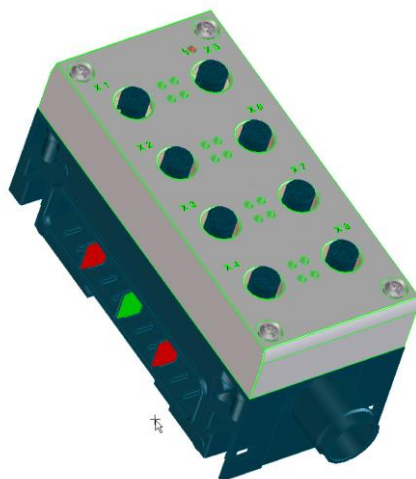
Még egy-egy modul felépítése is uniformizált: a táp- és buszvezetékek (sínrendszer) összekapcsolódását biztosító alépítménybe [3] rugós érintkező pontokon csatlakoznak a modulok tényleges funkcionálisát ellátó alsó, elektronikai egységek [2], melyekre felülről többféle ipari csatlakozási szabványhoz igazodó csatlakozó-feltét (M8 kisáramú, M12 nagyáramú, rugós gyorscsatlakozós, stb.) illeszthető [1].



A CPX modulok moduláris felépítése (csatlakozás – elektronika – CBUS)

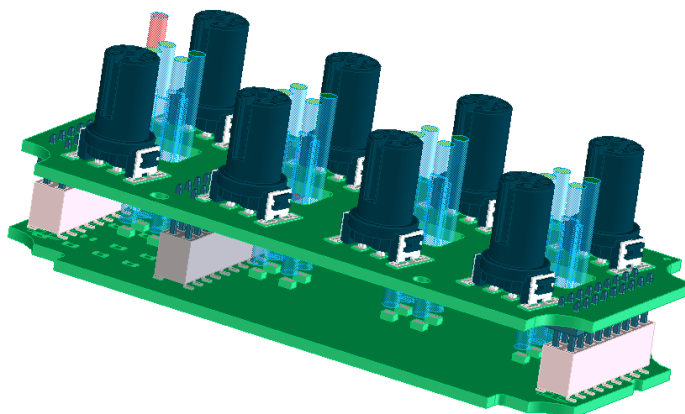
BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 22. oldal
--	--	--

A fejlesztési megbízás egy olyan elektronikus bemeneti egység kifejlesztése volt, amely 8 csatlakozón összesen 16 db 24V-os kétállapotú (digitális) jelet szolgáltató szensor túlterhelésvédett tápellátását biztosítja, fogadja a szenzoroktól érkező jeleket és azokat paraméterezhető előfeldolgozás után továbbítja a master egység felé. Felügyeli és jelzi a szenzorok tápellátásának túlterhelését is (visszakapcsolás/végleges lekapcsolás).



A CPX-16DE-D digitális bemeneti modul

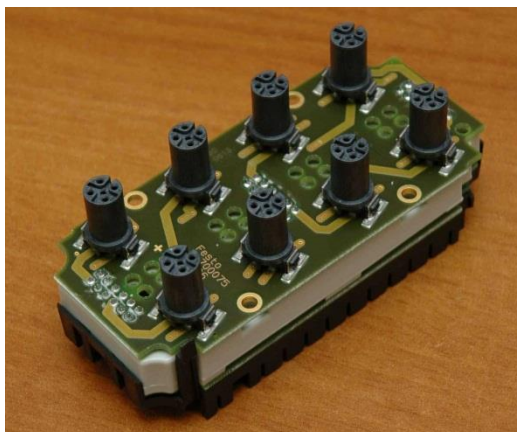
A modul konstrukciójában igazodik a korábban vázolt "rétegezett" felépítéshez (még a hardver tervezés beindítása előtt ilyen konstrukciós tervek készülnek egy-egy modul részletezett felépítéséről):



A 16DE-D modul konstrukciós terve

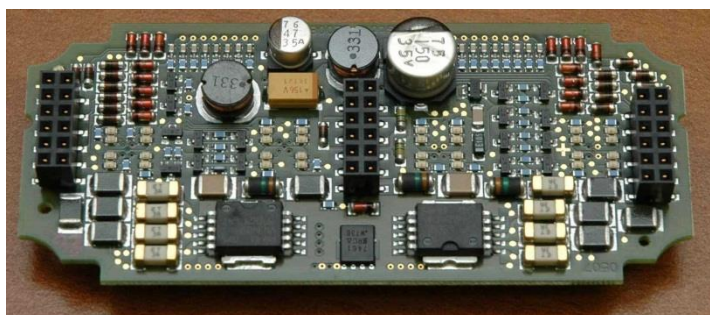
A terven jól látható, hogy a felső áramköri panel csak a csatlakoztatást szolgálja, még a kijelző elemek is az alsó, elektronikai egységen helyezkednek el, fényvezetők szolgálnak a kijelzés (zöld fény: bemenet aktív, piros fény: szensor táp túlterhelés) „kivezetésére” a felül található előlap felé. Szokjuk meg, hogy a mechanikai elemek sok tekintetben determinálják az elektronika kialakítását és elhelyezését is (nem ritkán azért kell a jól bevált alkatrészeinket más típusokra cserélni, mert azok a mechanikai felépítés következtében nem „férnek el” megfelelő helyen az áramköri panelen).

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 23. oldal
--	--	--

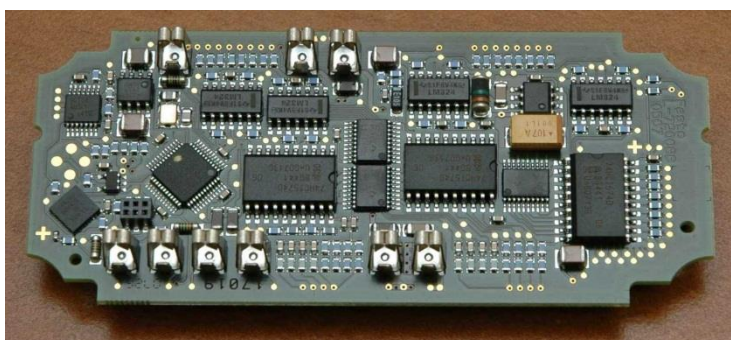


A modul fedél nélkül (felső panelen látszanak a fényvezetők helyei)

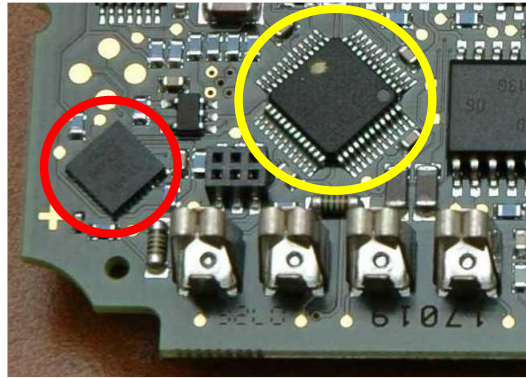
Kicsit lépünk előre az időben, és rögtön megmutatjuk a tervek kialakításakor még csak az elképzeléseinkben létező modul elektronikai paneljének végleges kialakítását is:



A tényleges elektronika (a felső, csatlakozó panel felőli oldal)

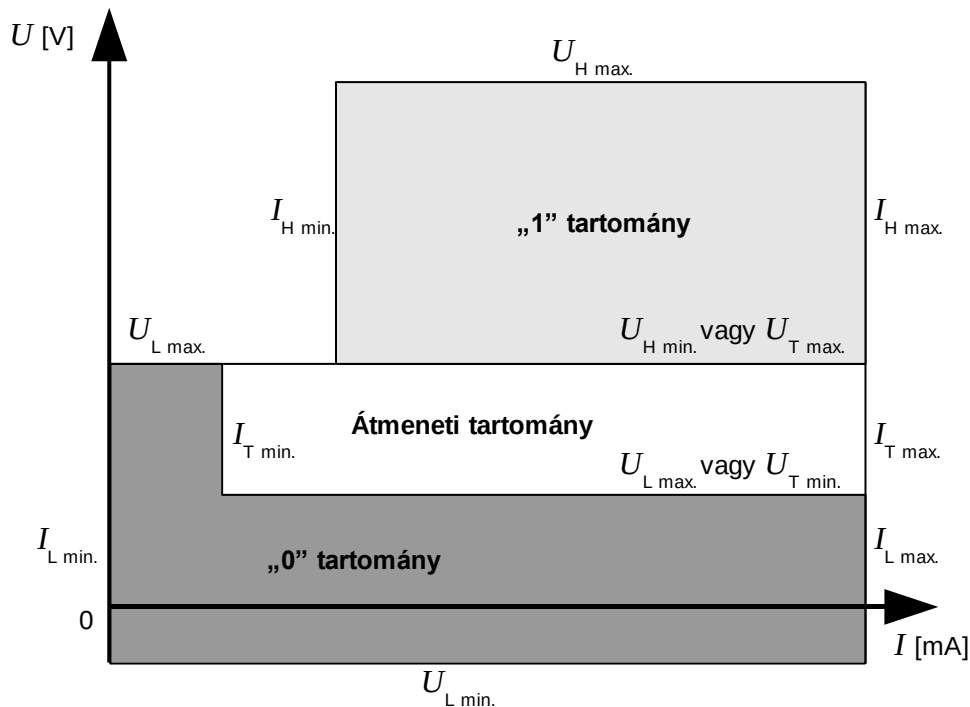


A tényleges elektronika (alsó oldal – jól látszanak a rugós buszcsatlakozók)



A mikroprocesszor (piros) és a buszkommunikációs ASIC (sárga)

De nézzük magát a megoldandó feladatot! A 16 db 24V-os, kétállapotú bemenet az IEC 61131-2 szabvány előírásainak megfelelően egy definit "0", egy jól definiált "1" és a kettő között egy ún. átmeneti tartománnyal kell rendelkezzen.



Digitális bemenet U-I tartományai (IEC 61131-2)

Bemenő jel	Limit	„0” állapot		Átmeneti tartomány		„1” állapot	
		U_L [V]	I_L [mA]	U_T [V]	I_T [mA]	U_H [V]	I_H [mA]
DC 24V	Max.	11/5	30	11	30	30	30
	Min.	-3	n.d.	5	2	11	6

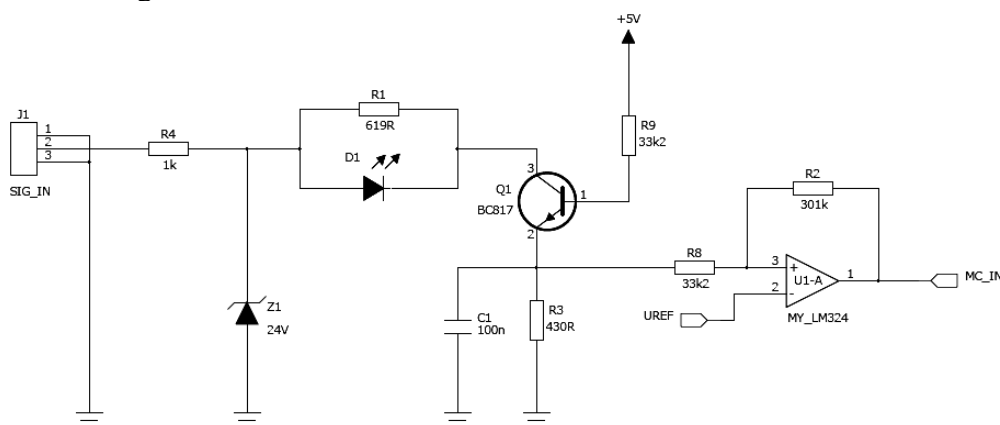
Digitális bemenet működési jeltartományai (IEC 61131-2; n.d. = Not Defined)

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 25. oldal
--	--	--

A diagram alapján azonnal látható a szabványalkotónak az a határozott törekvése, hogy

- Ameddig a bemenő feszültség értéke egy határt (U_{Lmax}) nem halad meg, bármilyen terhelő áram esetén is logikai „0” szintet kell felismernie az áramkörnek.
- A bemeneteknek (a védőáramkörök nyitóirányú feszültségese következtében) adott határig negatív bemenő feszültség (U_{Lmin}) esetén is helyesen kell működjenek.
- Kis bemenő áramok tartományában a bemenő feszültség magasabb értéke is még logikai „0” jelszintet takar (statikus túlfeszültségek).
- A logikai jeltartományok határozottan váljanak el egymástól, az átmeneti tartomány elkerülhetetlen, ez a sáv a kapcsolás számára bármelyik bemeneti jelszint azonosítását is eredményezheti.
- A logikai „1” jelszint ne lehessen véletlen műve – ehhez meghatározott terhelőáramnak kell társulnia
- a 24V mindig meghatározott toleranciasávok között értendő, az ipari alkalmazások esetében ez tipikusan $\pm 25\%$ (18..30V)

A fenti követelményeket teljesítő bemeneti fokozatot a következő kapcsolással valósítottuk meg:



Egy digitális bemeneti fokozat

A működés részletes elemzését az olvasóra bizzuk, egy-két, a megértést segítő megjegyzés:

- A bemeneti védőkapcsolás (R4-Z1) a kapcsolási túlfeszültségek és a fordított polaritású feszültség ellen nyújtanak védelmet.
- A speciális tranzisztorkapcsolás (Q1, pszeudo áramgenerátor) biztosítja a megkívánt áramterhelést, valamint azt, hogy a visszajelzés (D1) fénye lehetőleg ne változzon lényegesen a logikai „1” tartományban (kb. 15-30V között).
- A kimenőjel a tranzistor emitterkörében található munkaellenálláson keletkezik.
- A határozott billenést hiszterézises komparátor kapcsolás (U1-A) biztosítja.
- A mikrokontroller mintavételezésénél (100 μ sec – ld. később) gyorsabb jelátmenetek (alapvetően környezeti zaj) hardver úton történő szűrése (Q1 emitterkörében található R3-C1 szűrő).

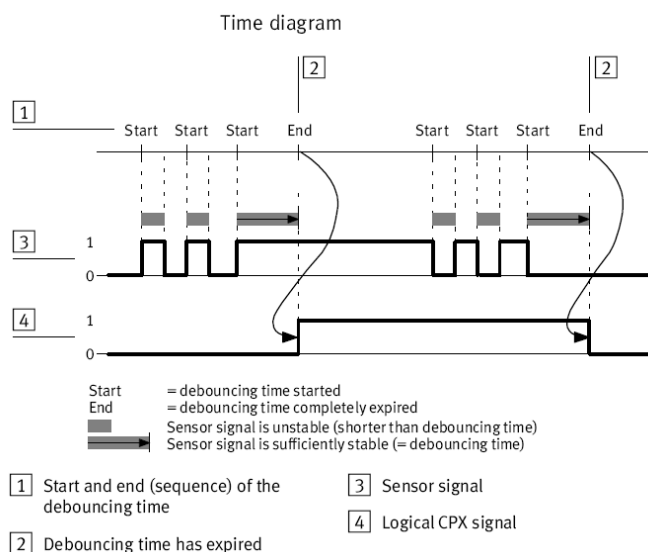
BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 26. oldal
--	--	--

A digitális bemenőjelek paramétereizhető jelformálás után kerülnek továbbításra, ezek a következők lehetnek:

- ✓ Pergésmentesítés: 0.1 ms, 3 ms, 10 ms, 20 ms
- ✓ Impulzushosszabbítás: 0, 0.5 ms, 15 ms, 50 ms, 100 ms

Mire jó (avagy miért kell) mindez ?

- Pergésmentesítés: általában ismert az alkalmazott tényleges kapcsolóelem, ennek pergési tulajdonságai alapján beállítható egy biztonsági felső időkorlát ahhoz, hogy ne legyen téves kapcsolásjelzés (viszont minél nagyobb értéket állítunk be, annál kisebb működési frekvencia lehetséges!)
- Impulzushosszabbítás: a kiszolgáló egység a felügyelő (host) számítógép ciklusidejéhez igazítja a beérkező jelváltozások tulajdonságait annak érdekében, hogy „ne vesshessen el” impulzus (ez szintén a működési sebesség korlátozása!)



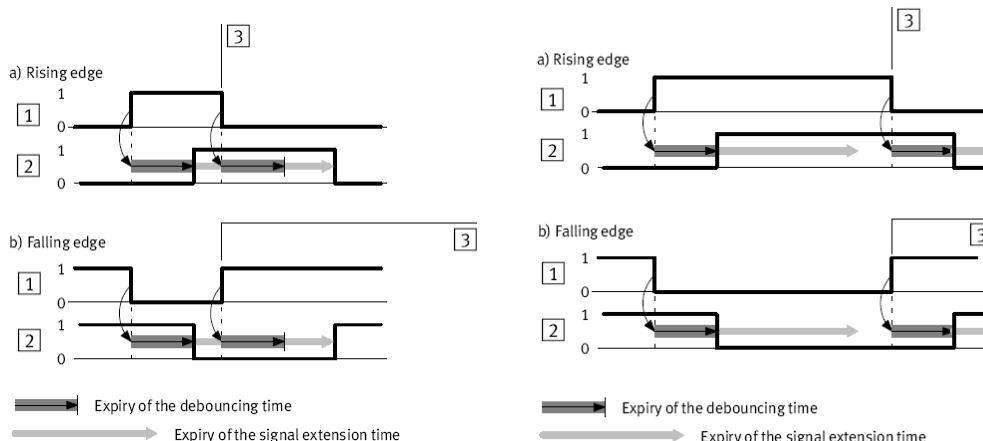
A pergésmentesítés elve

A pergésmentesítés során azt az egyszerű elvet követjük, hogy a bemenőjelet mintavételezzük, és amennyiben annak állapota előző állapotához képest nem változott, növeljük egy, a bemenethez tartozó számláló értékét. Változás esetén a számlálót nullázzuk. Amennyiben a számláló értéke elérte a paraméterben megszabott pergésmentesítési időt (pl. 100 μ sec $\times$ 30 = 3 msec), abban az esetben „elhiszük” azt, és jelezzük a bemenő jel megváltozását.

Az impulzushosszabbítás során a pergésmentesített bemenőjel impulzusok hosszát (amik nem lehetnek rövidebbek, mint a beállított pergésmentesítési idő) meghosszabbítjuk a megadott értékre. Két esetet kell megkülönböztessünk:

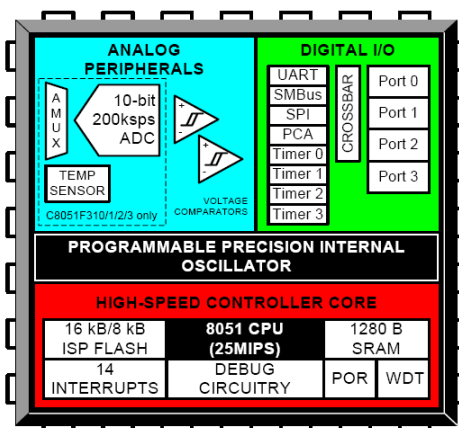
1. amennyiben a bemenőjel „visszaváltása” hamarabb következik be, mint a megadott legrövidebb impulzusidő, akkor azt meg kell hosszabbítanunk a megadott időre
2. amennyiben a bemenőjel-impulzus eleve hosszabb volt, mint a megadott legrövidebb impulzus ideje, akkor csak a pergésmentesítés leteltét kell kivárnunk.

(Vegyük észre, hogy az előállított impulzus hossza semelyik esetben sem lehet rövidebb, mint a beállított pergesmentesítési idő!)



Az impulzushosszabbítás elve (1. és 2. eset)

A feladat nem látszik túl bonyolultnak, mindenki számára megoldhatónak tűnik a jelfeldolgozás egy parányi (5x5 mm) és olcsó (nagyobb darabszámban kb. 3€/db) mikrokontrollerrel. Különösen a paraméterezett időzítés/számlálás lenne bonyolult egy esetleges analóg vagy digitális, de hardver megoldással. A kiszemelt mikrokontroller - elsősorban a viszonylag nagy művelet végrehajtási sebesség (átlagosan 15-20 MIPS) és a bitkezelést támogató belső (8051-es) architektúra miatt - a Silicon Laboratories C8051F311 típus lett.



A C8051F311 mikrokontroller (MLP-28 5x5 mm tokozásban)

Első és legfontosabb kérdésünk az volt: a lehető leggyorsabban működtetve a mikrokontrollert "elbírjuk-e" 16 bemenet független feldolgozását annak figyelembevételével, hogy a legrövidebb előírt időkvantum 100 µsec (a legrövidebb pergesmentesítési idő), miközben a mikrokontrollert belső órajel-generátoráról a lehető legnagyobb sebességgel ($f=24.5 \text{ MHz} \Rightarrow T=40.82 \text{ nsec}$) működtetjük?

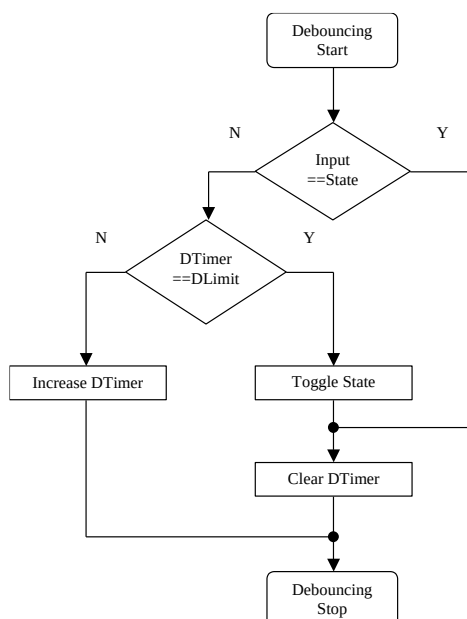
A legfontosabb alapelvek a következők:

- A 16 fizikai bemenet 2 db 8 bites portra rendezhető, ezeket 100 µsec-ként beolvassuk (a szenzortáplálás kapcsolása + diagnózis és az SPI kommunikáció miatt a 28 láb nem elegendő arra, hogy a bemeneteket közvetlenül a portlábakra csatlakoztassuk)
- A beolvasott értékeket (2 bájt) letesszük a bitcímezhető területre ⇒ ezzel valamennyi bemenet direkt módon címezhetővé válik
- Ciklust nem szervezünk, mert
 - DJNZ időbe kerül ($4T \times 16 = 64T = 2.6 \mu\text{sec}$)
 - Indirekten kellene címezni mind a biteket, mind azok számlálót, ami egyértelmű idővesztés lenne
 - ⇒ lemásoljuk a feldolgozást 16-szor egymás mögé!

CLR	bInputLEN	// Select Input0-7	
MOV	ucInputL, P1	// Read the inputs	(3)
SETB	bInputLEN	// Deselect	(2)
CLR	bInputHEN	// Select Input8-15	(2)
MOV	ucInputH, P1	// Read the inputs	
SETB	bInputHEN	// Deselect	

A bemenetek beolvasása

A beolvasás egyidejűségi hibája a kézzel jelzett utasítás végrehajtási periódusidők alapján $7T = 286 \text{ nsec}$. A bemutatott algoritmus és a hozzátartozó kódrészletek elemzését az olvasóra bízunk.



A pergésmentesítés folyamatábrája

```

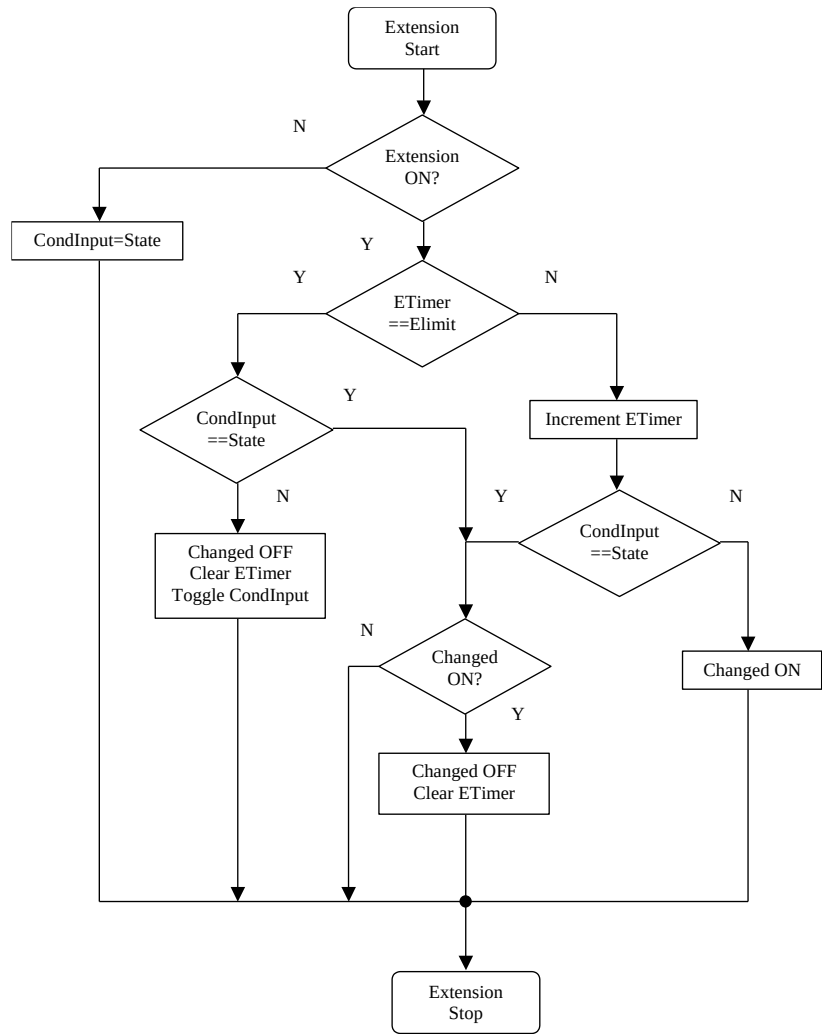
L0_Deb:      MOV     C,bInput0           // if (bInput==bState)
             JNB     bState0,L0001
             CPL     C
L0001:      JNC     L0002
             MOV     A,R0
             CJNE    A,ucDcount0,L0003
             CPL     bState0           // if(bInput==bState) -> FALSE
             // if (ucDlimit==ucDcount) -> TRUE
             // Toggle bState

L0002:      MOV     ucDcount0,#00H      // if(bInput==bState) -> TRUE
             SJMP    L0_Ext            // Clear Debouncing counter
             // jmp Signal Extension

L0003:      INC     ucDcount0           // if(ucDlimit==ucDcount) -> FALSE
             // Increase Debouncing counter
L0_Ext:

```

A pergésmentesítést megvalósító programrészlet



Az impulzushosszabbítás algoritmus

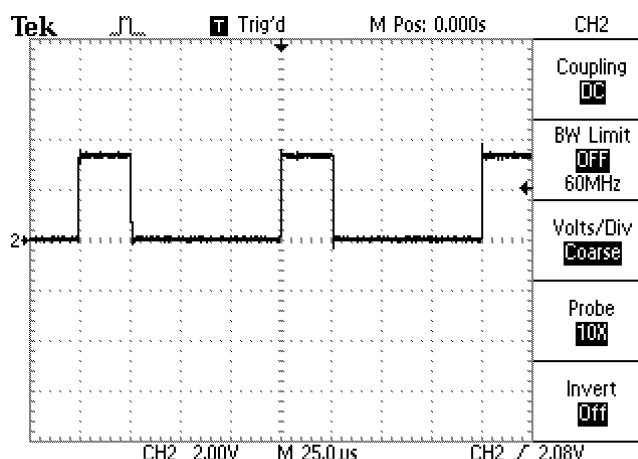
BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 30. oldal
--	--	--

L0_Ext:	JNB	bExtension0,L0004	// if (bExtension==1)
			// if(bExtension==1) -> TRUE
	MOV	A,R2	// if (usEcount==usElimit)
	CJNE	A,usEcount0+01H,L0005	
	MOV	A,R1	
	CJNE	A,usEcount0,L0005	
			// if(usEcount==usElimit) -> TRUE
	MOV	C,bState0	// if (bCondInput==bState)
	JNB	bCondInput0,L0006	
	CPL	C	
L0006:	JNC	L0010	
			// if(bCondInput==bState) -> FALSE
	CLR	bChanged0	// Clear cChanged
	CLR	A	
	MOV	usEcount0,A	// Clear usEcount
	MOV	usEcount0+01H,A	
	CPL	bCondInput0	// Toggle bCondInput
	SJMP	L0_Ext_Stop	// Exit from Signal extension
L0004:	MOV	C,bState0	// if(bChanged==1) ->TRUE
	MOV	bCondInput0,C	
	JNC	L0_Ext_Stop	
L0009:	SETB	bChanged0	// if(bCondInput==bState) -> FALSE
	SJMP	L0_Ext_Stop	
L0005:			// if(usEcount==usElimit) -> FALSE
	INC	usEcount0+01H	// Increment usEcount
	MOV	A,usEcount0+01H	
	JNZ	L0007	
	INC	usEcount0	
L0007:	MOV	C,bState0	// if (bCondInput==bState)
	JNB	bCondInput0,L0008	
	CPL	C	
L0008:	JC	L0009	
			// if(bCondInput==bState) -> TRUE
L0010:	JNB	bChanged0,L0_Ext_Stop	// if (bChanged==1)
			// if(bChanged==1) ->TRUE
	CLR	A	
	MOV	usEcount0,A	// Clear usEcount
	MOV	usEcount0+01H,A	
	CLR	bChanged0	// Clear bChanged
L0_Ext_Stop:			

Az impulzushosszabbítást megvalósító kódrészlet

Fenti kódrészleteken gyakorlott programozók dolgoztak kb. két munkanapon át. A kitűzött cél az volt, hogy hogyan lehet a fenti kódrészleteket a lehető legrövidebb idő alatt végrehajtani a mikrokontrollerrel. A folyamat során folyamatosan számolták az órajel-periódusokban mért végrehajtási időt, mint a megvalósíthatóság egyik alapfeltételét.

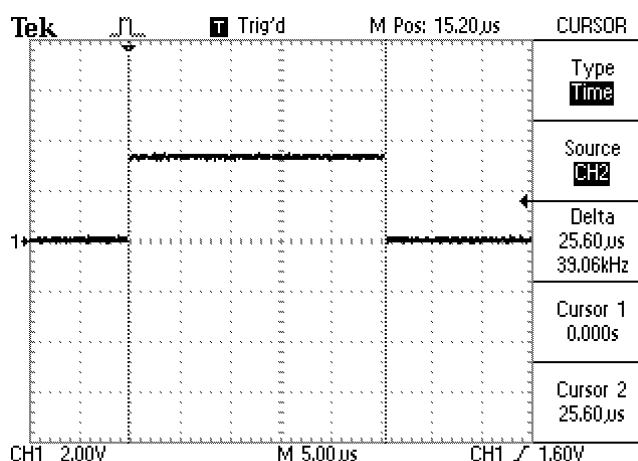
Természetesen a számított értékeket mérésekkel ellenőriztük is. A szoftver működését mérésekkel ? Igen – ez egy nagyon fontos lehetőség, ha csak tudunk, éljünk vele minden esetben ! A ténylegesen beprogramozott kódrészlet egy 100 µsec időzítéssel futtatott megszakítási rutinba került, a programrészlet elején egy kimenetet „1”-be állítottunk, majd a kódrészlet lefutása után ismét „0”-ba került a kimenet. Az oszcilloszkópon a következő kép volt megfigyelhető:



A programrészlet futásidejének ellenőrzése oszcilloszkóppal

És akkor az első tényleges ellenőrzés: a mikrokontroller 24.5 MHz órajel-frekvenciával működik, azaz egy órajel periódusideje $T=40.82$ nsec. Nyugalmi helyzetben (a bemeneteken nincs változás) a pergésmentesítési algoritmus 15T (612 nsec) időt igényel, az impulzushosszabbítási programrészlet 22T (898 nsec) alatt hajtható végre bemenetenként. A megszakítási rutin további 37T „hulladékidőt” igényel a regiszterek mentése/visszaállítása, a bemeneti portok beolvasása céljára. Ezzel nyugalmi helyzetben tehát a következő eredményre jutottunk:

$$(37 + 16 * (15 + 22)) * 40.82 \text{ nsec} = 25.68 \mu\text{sec}$$



A feldolgozás futásideje állandó bemenetek esetén

Az oszcilloszkóp képernyőjének jobboldali középső mezőjében jól látható az igen jó egyezés a számított értékekkel.

Ezt követően már bonyolultabb feladat következett: a bemenetek változásai függvényében lehetséges „futásidő-kombinációkat” számoltunk abban az esetben, ha különböző időpontokban, de valamennyi bemenet egy időben mozog. A

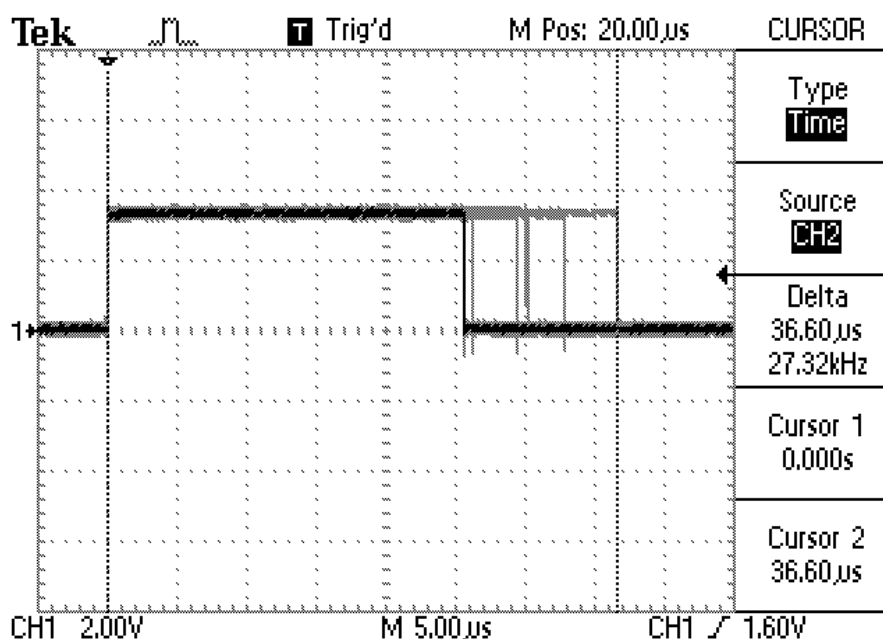
BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 32. oldal
--	--	--

programrészletek független volta miatt ez a feltételezés nyugodtan megtehető, a legrosszabb eset (worst case) így is megfigyelhető kell legyen.

1 csatorna [T]	16 csatorna [T]	16 csatorna + IT-kezelés [T]	Az IT rutin számított hossza [µsec]	Megjegyzés
37	592	629	25,68	Nyugalmi helyzet
38	608	645	26,33	
43	688	725	29,59	
44	704	741	30,24	
48	768	805	32,86	
54	864	901	36,78	

Futásidők különböző állapotokból indított algoritmusok esetén

A most megfigyelt idődiagram a következőképpen alakult:



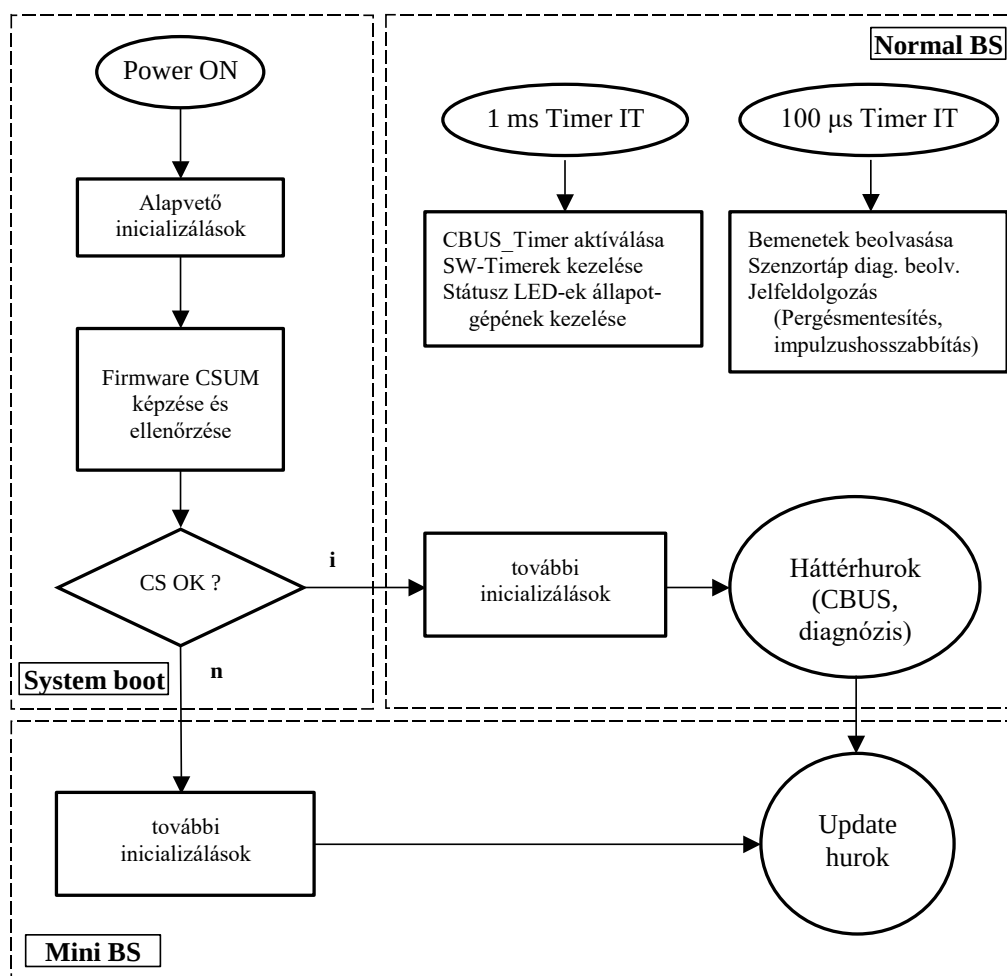
A feldolgozás futásideje változó bemenetek esetén

A számított és a mért idők ismét megnyugtató egyezést mutattak. Ezek alapján a mikrokontrollerre a jelfeldolgozás várhatóan $37 \mu\text{sec} / 100 \mu\text{sec} = 37\%$ terhelést jelent, ami nyugodtan vállalható a környező események lényegesen lassabb volta miatt (tápellátás-figyelés kb. 50-100 msec-ként, adattovábbítás a master egység felé kb. 1 msec-ként).

Fentiek alapján már össze is állíthatjuk firmware rendszerünk működési blokkvázlatát:

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 33. oldal
--	--	--

1. Minden indulás (Power-ON) után a rendszerünk önellenőrzést végez. Csak az operációs rendszer hibátlan volta (külön helyre letárolt ellenőrző összeggel való összehasonlítás után) indítjuk a rendszer működését.
2. Hibás rendszer detektálása (vagy a működés alatt kapott ilyen értelmű parancs) esetén egy biztonsági „Mini-BS” kerül aktiválásra, ami csak a firmware rendszer törlését és update-elését teszi lehetővé.
3. A rendszerben a legmagasabb (senki által nem tiltható) prioritáson fut a 100 μ sec-os IT rutin, ami bármit megszakíthat és max. 40 μ sec-ra felfüggesztheti annak működését.
4. A „lassabb” időzítési feladatokat (mint a túlterhelés figyelő rendszer 50-100 msec-kénti működtetését, a mért adatok 1 msec-kénti továbbítását, a rendszer timeout-ok kezelését, a hibaállapotokat jelző ledet 1-2 Hz-es villogtatását) egy alacsonyabb prioritású 1 msec-os időzítő rutin végzi)
5. A nem időkritikus feladatok (további rendszerállapotok figyelése, CBUS kommunikációs tesztparancsok, stb.) a háttérhurokba kerültek.



A CPX-16DE-D modul szoftver rendszerének blokkvázlata

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2b.DOC 2023.10.13. Dr. Tevesz Gábor II / 34. oldal
--	--	--

Foglaljuk össze, mit is csináltunk eddig a feladathoz kapcsolódóan ?

1. Megismerkedtünk annak a rendszernek a tulajdonságaival, amelyhez a fejlesztésünk során kapcsolódnunk kell
2. Részletesen megismertük a feladatot, amit meg kell valósítanunk (annak a „miértjeit” is, azaz semmit ne fogadjunk el csak azért, mert „azt mondták”).
3. Elképzeltünk egy hardver-szoftver feladatmegosztást. Korábbi fejlesztési tapasztalataink alapján a kapcsolások „durva” részletei szinte azonnal felvázolhatók a feladathoz – a méretezés természetesen egy későbbi fejlesztési stádium feladata lesz.
4. Némi gyakorlattal azonnal érezhető, mi lesz a szoftver kritikus része, ahol a bonyolultság (memóriaszükséglet) vagy a gyorsaság (utasítások végrehajtási sebessége) esetleg szűk keresztmetszet lehet.
5. Ezekhez a programrészekhez egy valóságos (fejlesztő kitek) vagy szimulált környezetben teszt kódokat készítünk. Gondoljunk arra, hogy az itt „elfecsérelt” órák napok, hetek rossz irányba történő fejlesztési ráfordításait előzhetik meg !
6. A teszt kódokat mérésekkel ellenőriztük (méret, sebesség).
7. Fentiek alapján már össze tudtuk állítani azt a globális blokkvázlatot, aminek megfelelően – team munka esetén – akár több párhuzamos szálon is elindítható a fejlesztési munka.

Az összeállított **rendszerterv** alapján az egység hardver-szoftver elemeinek tényleges tervezése és megvalósítása elkezdődhet – a modul várhatóan valamennyi elvárásunknak meg tud majd felelni. A folyamat így módon le is zajlott, a bemeneti egység jelenleg több tízezer példányban működik a Festo ipari automatizálási rendszereiben.